Jan-Øivind Lima

Developing a constraint satisfaction procedural music generator using artificial intelligence to evaluate its effectiveness as a development tool

Generating meaningful artistic content using simple procedural generation methods developed using an iterative development strategy assisted by artificial intelligence

Master's thesis in Cybernetics and Robotics
Supervisor: Sverre Hendseth
June 2026

Norwegian University of Science and Technology
Faculty of Information Technology and Electrical Engineering
Department of Engineering Cybernetics

NTNU

ABSTRACT

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

SAMMENDRAG

Write an abstract/summary of your thesis, and state your main findings here.

A summary should be included in both English and any second language, if this is applicable, regardless if the thesis is written in English or in your preferred language. These should be on separate pages, the English version first.

PREFACE

Write the preface of your thesis here.

You may include acknowledgements and thanks as part of your preface on this page, or you may add it as a new chapter after the preface.

This thesis is written as the final master project at Norwegian University of Science and Technology (NTNU) during the spring of 2026, supervised by Sverre Hendseth.

CONTENTS

LIST OF FIGURES

LIST OF EXAMPLES

All examples can be found at my github.io page where they can be listened to, numberings are consistent with the thesis for convenience.

LIST OF TABLES

ABBREVIATIONS

List of all abbreviations in alphabetic order:

AI	Artificial Intelligence
AC	Arc Consistency
BSP	Binary Space Partitioning
CS	Constraint Satisfaction
LISP	List Processing
LLM	Large Language Models
MIT	Massachusetts Institute of Technology
NTNU	Norwegian University of Science and Technology
OOP	Object Oriented Programming
PG	Procedural Generation
PCG	Procedural Content Generation
WFC	Wave Function Collapse
QM	Quantum Mechanics

CHAPTER

ONE

INTRODUCTION

For hundreds of years composers have been creating music using external tools like dice to help them generate something new. As early as the 18th century the idea of using randomness to help the composing process has been used. The musical dice game, "Musikalisches Würfelspiel", was used to randomly generate music based on pre composed parts. [1] This is a very basic way of generating music, however by the random nature of the dice it will not be able to create anything coherent and meaningful like a human composer can. Additionally this process only used pre composed parts which were designed to work together. If we want to generate everything from scratch we need a way more sophisticated system. This is where procedural generation comes into the picture. If we base the melody on something random and then use the conventions and "rules" for melodic composition we might be able to generate music that sounds somewhat coherent.

Getting external help has not only been used by composers, but also by programmers. In recent years the use of AI in software development has become more and more widespread. As an experiment in this thesis I will compare "old-school" development and AI. I will start with normal development without using AI. Then after I have build up some knowledge with the system I will try to improve upon it using AI tools and evaluate both the output of the code and code itself.

Today the most common way to go about generating music is using AI and Large Language Models (LLM), but what goes on inside these large models is not easy to understand and tweaking them for a specific use case is not trivial. Using such models enables anyone to create music, however these models are trained on large data sets and imitates what has come before not generating something truly new. For this thesis the focus is on the craft of creating music, breaking it down to its very primitive form, only pitches and durations. This will enable the user to have more control over exactly what is being generated. However this requires us to create a system and models that capture the essence of how exactly we are to go about constructing the very basic musical ideas. It will not depend on large data sets and will create a musical ideas that can be processed into a fully fledged musical idea. This task is not trivial since the rules of creating music is not absolute and the innate human element of creativity needs to be done by a

computer.

If we are able to solve this problem of creating a good structure and model for this kind of generation we will not only have a solution for musical ideas, but this kind of system could solve many kinds of creative problems.

The problem we want to solve is the problem of creating a coherent system that will be able to generate music that adheres to certain musical principles. In order to do this we need to consider many things, and system design is a big part of this. When building such a program we need to consider what parts it should consist of as well as how they are supposed to interact. Object Oriented Programming (OOP) is a solid way to approach this system design. Additionally I will explore how the use of AI in software development can aid in solving problems and speeding up the process.

The ideal solution to this problem would be a system that not only is able to solve this problem of creating music that is self similar and coherent, but is constructed in such a way that the solution can easily be generalized in order to solve other problems. The problem of generating something coherent is not special to only music, but has applications for many other domains, especially creative ones or where there are a large number of usable solutions. For instance ways to connect cities by roads.

The thesis is structured as follows. Firstly there is some background to give some context. This background includes music theory, this section is just to give the reader a basic insight in how music is constructed and how notation works in order to understand figures and examples that show up throughout the thesis. Following is a section about procedural generation, this section explains a few methods of procedural generation briefly to give context about how a few methods work as well as some illustrative examples on how they can be utilized. After this there are a few brief sections about object oriented programming and the use of AI in software development. Then comes the bulk of the thesis with the iterations of the software. The three iterations are presented as their own complete system.

CHAPTER

TWO

BACKGROUND

This chapter provides the background theory for the project. First we give an overview of procedural generation and some of the different methods used in procedural generation. Then we give an overview of some basic music theory that is necessary to fully grasp the structure of the system. Lastly we cover some about object oriented programming.

2.1 Procedural generation

Procedural Content Generation (PCG), often reffered to as just Procedural Generation (PG), is algorithmic generation of content. This content can take many forms including landscape, levels, textures or even 3D models. This leads to smaller file sizes since the program can generate the content using algorithms rather than having everything pre-made and saved.

This section will cover several methods of PG and discuss similarities, differences and most importantly limitations of the methods. Additionally we will take a look at and what we are trying to achieve with the procedural generation in this project and propose a new method that will handle the problem of music generation in a new way.

2.1.1 L-systems (Lindenmayer systems)

WIP section

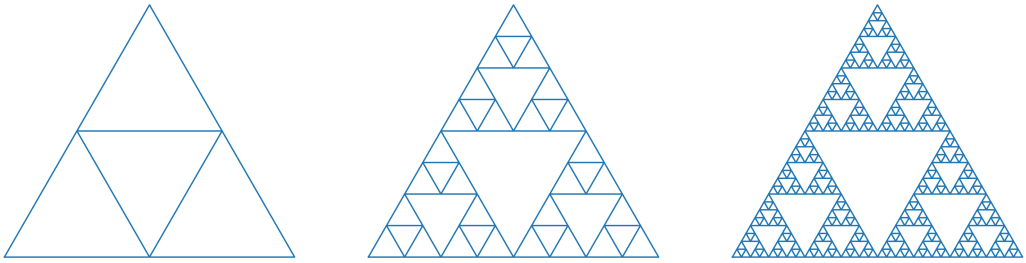
Lindenmayer systems are a method of PG based around the idea of creating rules for the generation which lead to self-similarity and fractal-like forms. This kind of PG is often used in generating organic structures such as trees or other plant-like structures.

If we observe Figure 2.1.1 we can see how two different L-systems can create both very rigid results in the form of the sierpinsky triangle and more organic structures like the fractal plant on the second line.

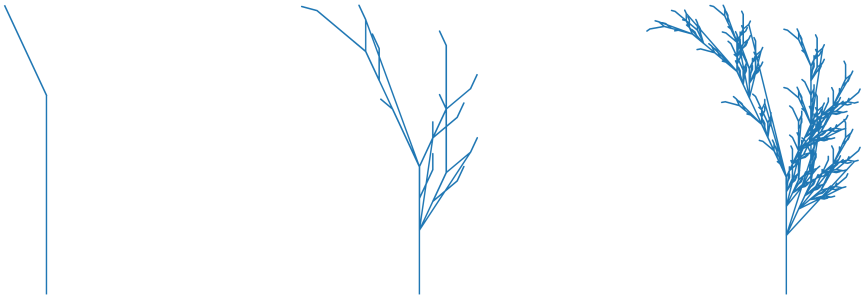
2.1.2 CS

WIP section

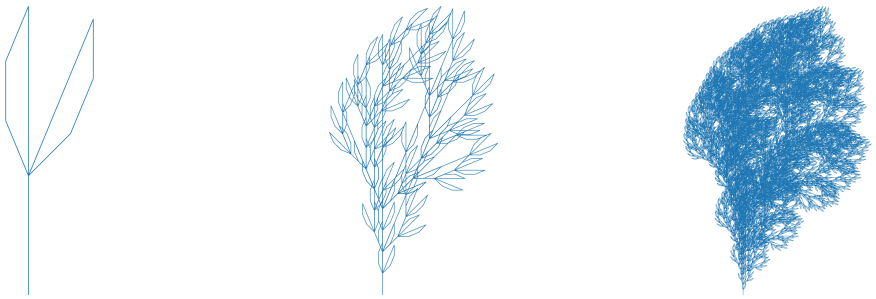
Similar to WFC we have PG using CS. This method uses given constraints for



(a) sierpinksi triangle



(b) fractal plant



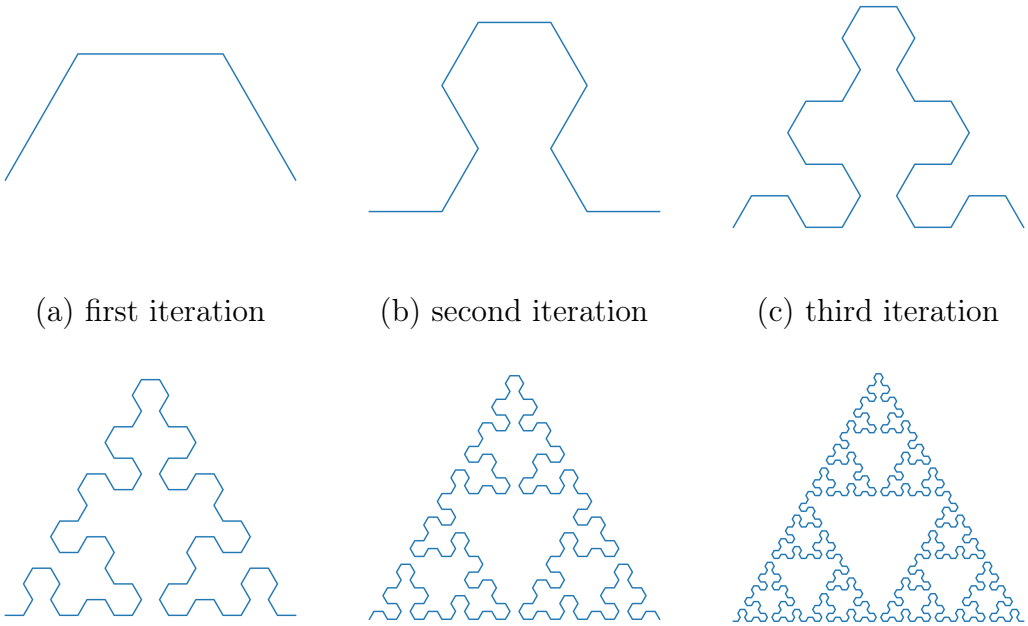
(c) fractal bush

Figure 2.1.1: Demonstration of the sierpinski triangle, a fractal plant and bush after one, three and five iterations repsectively.

influencing the generation. It usually works by randomly generating something in the given domain, then iterates over the generated data and enforces constraints in order to make it adhere to a model designed by the programmer or end user. After several iteration over the domain we end up with a domain where the constraints are satisfied for all the objects in the domain.

In Figure 2.1.3 you can see how the constraints are set up for this example, and Figure 2.1.4 shows the initial domain with how many different states are left for each cell in the domain as well as the domain after AC-3 propagation. Finally Figure 2.1.5 shows the final domain that satisfies both the local and the global constraints.

The Arc Consistency (AC)-3 algorithm reduces the search space by removing the states that are not possible [2]. If we look at the nodes in Figure 2.1.4 and imagine that for a cell it can be either *water* or *sand*. Then the neigbours can be either *water*, *sand*, *grass* or *path*. It can never be *forest*, *settlement* or *mountains*.



(a) first iteration (b) second iteration (c) third iteration

(d) fourth iteration (e) fifth iterations (f) sixth iteration

Figure 2.1.2: Approximation of the sierpinski triangle using the sierpinski arrowhead curve.

This is what you can see has changed between the two domains in the figure.

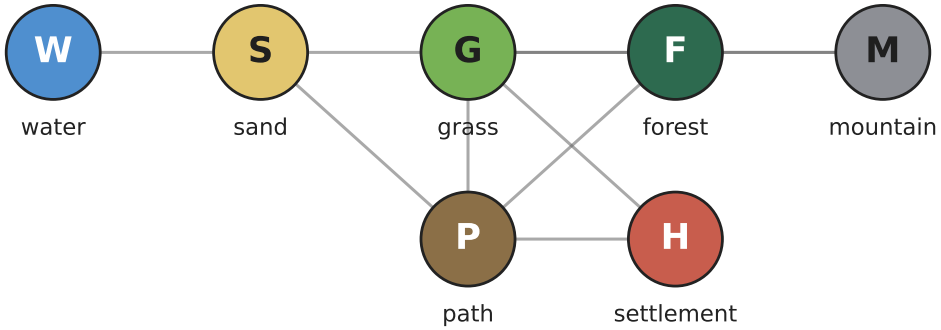


Figure 2.1.3: All different node types and how they are constrained. Each edge represents which note types can be adjacent in the domain.

2.1.3 WFC

WFC is a method used to procedurally generate using either given constraints, or it can also use some input that it analyses and generates the constraints procedurally which are then used to generate something in the same style as the input.

It works by initializing the entire space in a superposition which means that the state is yet to be determined. We call this an *unobserved* state, meaning

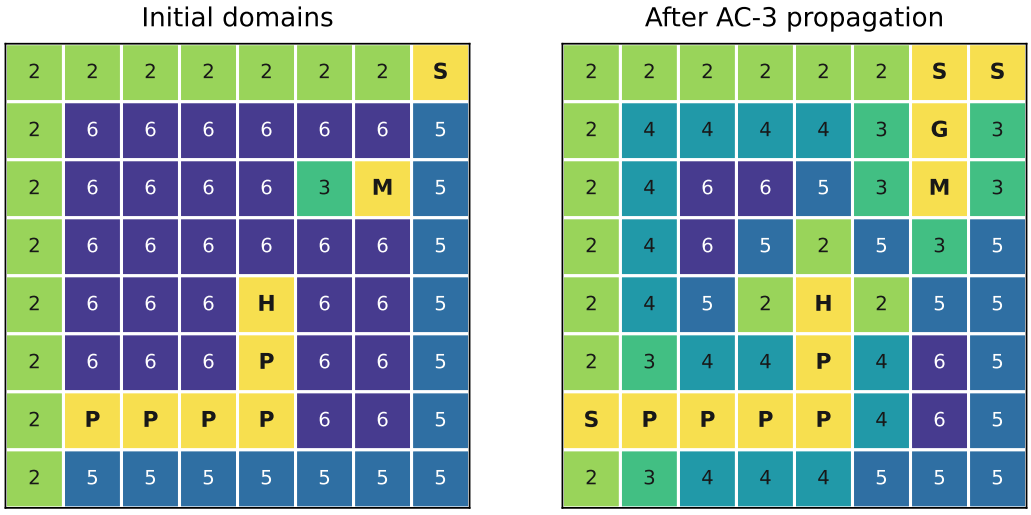


Figure 2.1.4: Initial domain and domain after AC-3 propagation. The AC-3 algoriths reduces the search space by ensuring arc consistency.

we don't know what state it is in yet. These unobserved states are not complex numbers like they are in Quantum Mechanics (QM), but the process is similar which is why it gets its name inspired by the field of QM. Randomly assigning parts of the space we are able to collapse[1] some of the *wave function*[2].

The rules for how the tiling should propagate are derived from an example input. In Figure 2.1.6 we see how from the simple input image the algorithm determines some local tiling rules for the different states and uses this as a basis for generating the new content. These generated rules are what make the output consistently similar and ensures we are outputting something that resembles the original input. This is the main difference between WFC and CS, for CS problems we need to design the constraints ourselves, but for WFC we only give an input and the algorithm derives the rules from the input.

In Figure 2.1.7 we can see how the number of possible states reduces as soon as a state is collapsed. This propagets outward and the process of assigning states continues until the entire domain has collapsed.

When the rules for the tiling have been established we can use them to generate new examples, in Figure 2.1.8 we are able to generate a larger domain using the same rules we got from the example image.

Finally we can see how the rules does not make it so that there is simply one output, we can change the seed for generation and end up with very different results, but all adhering to the initial rules derived from the exemplar in the very first step.

[1] collapsing a superposition means that we decide the actual state of the given position in the space

[2] the wave function is simply the constraints what decide what the state of the object should be dependent on external factors

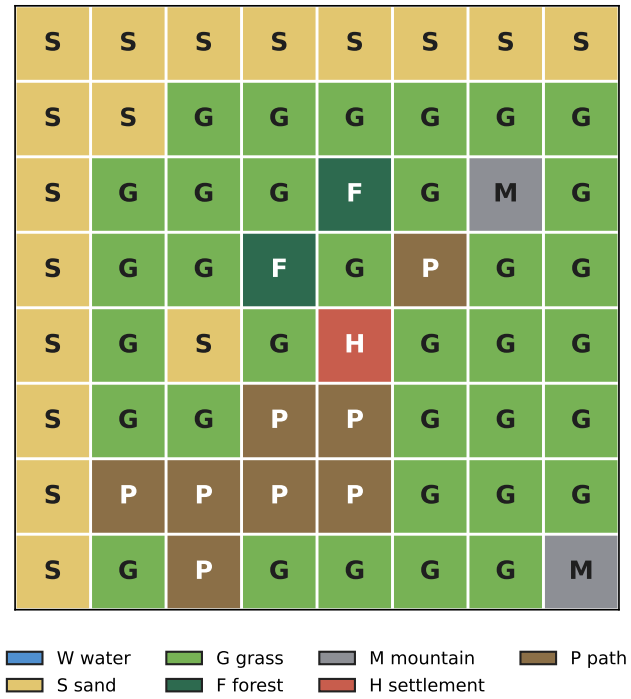


Figure 2.1.5: Final generated map satisfying all constraints.

2.1.4 BSP

WIP section
BSP can be used to generate map layouts like indoor environments, in the following figures you can see an example of how we can divide the entire domain into sections and used that to generate rooms.

2.1.5 Random noise and seeds

WIP section
Perlin noise is a very common way to generate structures like worlds etc... It is very good at creating smooth transitions from one state to another, like height maps. This is demonstrated in the following figures below with some different techniques.

2.1.6 Random walks

Random walk procedural generation method that can be used to organic looking structures like maps and textures. It can also be used to describe natural phenomena like bacterial movement. It has been applied in financial economics and computer science

The random walk works by having an entity walk around randomly. In some cases the random walk is also called the *drunkard's walk*. Then the path is traced, either to create a circumference of an area or the traversed spaces become the area itself. Oftentimes the drunk walker has a set amount of time or steps it can walk

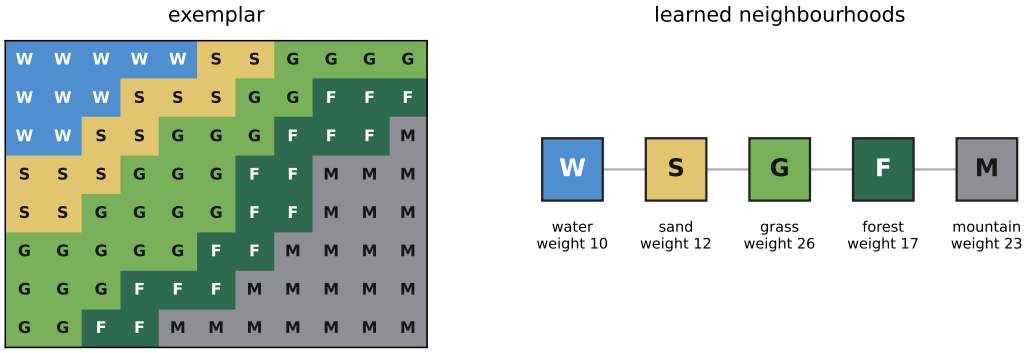


Figure 2.1.6: Given the exemplar on the left the WFC algorithm learns the local tile rules shown on the right.

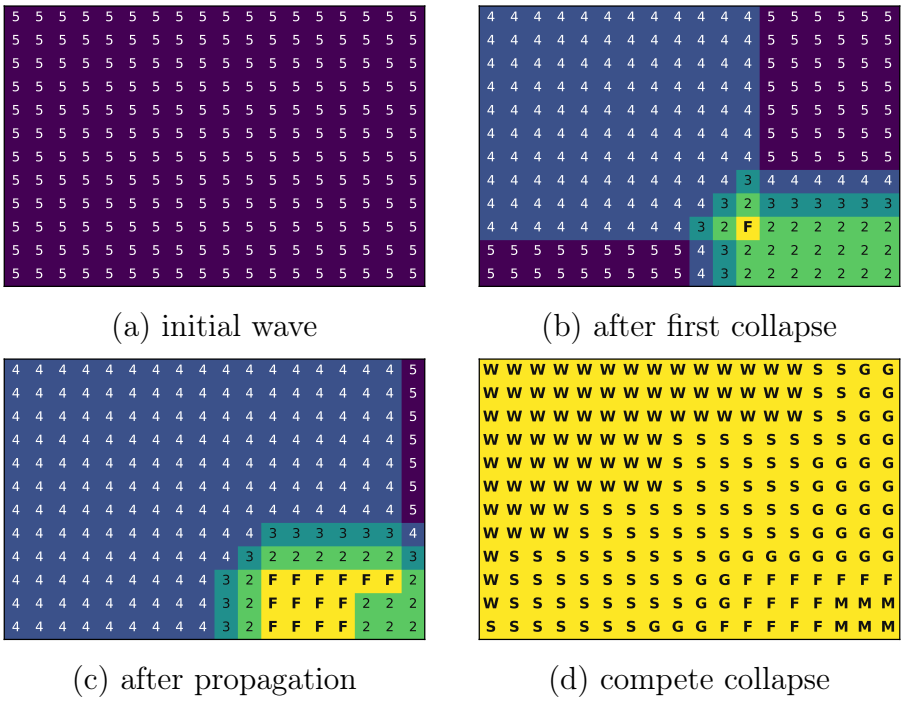
before it is terminated, then a new walker is placed at a random location, this helps to smooth out the generated area.

The same technique can be used to create textures as well, in Figure 2.1.16 we can see how changing the colours and directional bias we can generate textures that imitate real life. In the tree and sand example the directional bias is vertical for the tree texture and diagonal for the sand texture. For the map texture it is uniform directional bias, but colouring to indicate water, land and mountains. The least traveled to square get more blue, and the more frequent it goes to green and brown/white to indicate mountains.

2.1.7 Discussion

WIP section

Now that we have taken a look into a few methods of PG we observe that there are many similarities between the methods. Common for all forms of PG is that they utilize some form of randomness. The randomness in a core part of procedural generation, if there was no randomness we would generate the same output every time and that would be the same as just creating the content yourself.



(a) initial wave (b) after first collapse

(c) after propagation (d) compete collapse

Figure 2.1.7: Shows how collapsing/observing a state propagates. An observation reduces the local uncertainty.

2.2 Music theory

This section contains some basic music theory relevant to understand how music is constructed. It is not neccessary to understand everything in depth, but farmiliarity with these concepts will be of great help to understand the project as a whole. Throughout the thesis there will be several examples and figures using musical notation so a basic understanding of how to read these figures will be benefitial.

2.2.1 Notes

The very core of music is composed of notes. Notes are vibration in air which we interpret as pitches. In the western system we agree upon the standard of pitches is based around A_4 which is $440Hz$. All other pitches are tuned in relation to this reference.

2.2.2 Note values

The same musical pitch can be written in several different ways in order to dictate the duration of the note. This is done by using variations of the note. This can be by Refering to Figure 2.2.1 we can see the different parts of the note highlighted with colour. The note-head is coloured red and the placement of this decides which note is represented. The stem can go either up or down, this does not change the meaning it just makes it more compact to make it go down if the note placement is high on the staff. The direction of the stem can carry meaning if there are two notes simultaneously on a staff, then it usually indicates two different voices.

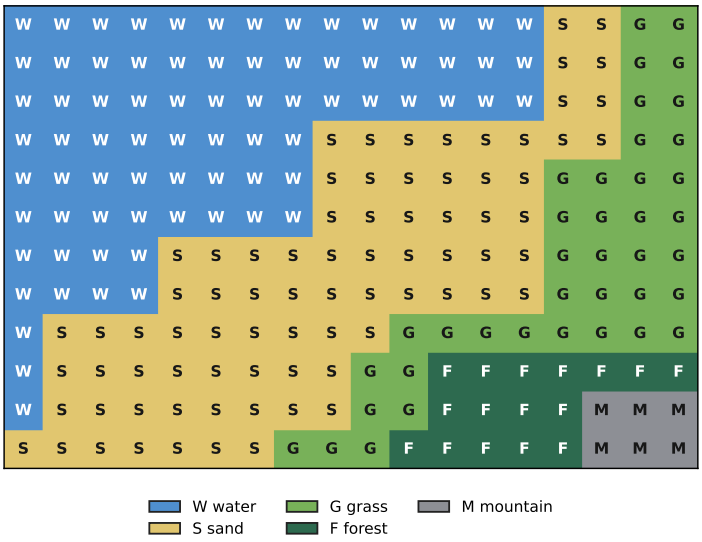


Figure 2.1.8: An example output for a larger domain than the original exemplar. The consistent rules lead to consistently similar outputs.

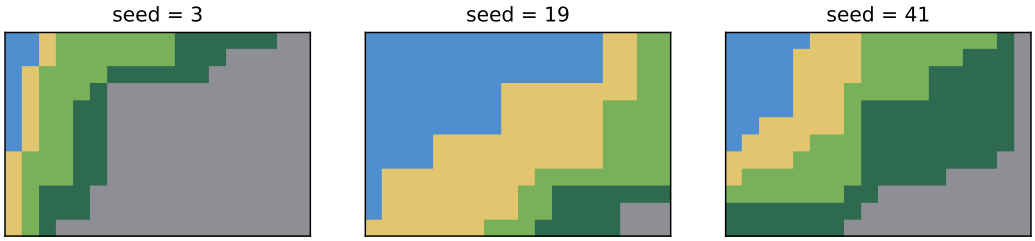


Figure 2.1.9: Illustration that the learned rules stay consistend while the seed changes the output.

Meaning one player playes the melody with stem up and one with the stem down. Next we have the beams and the flags coloured in purple and blue respectively. These two carry the same meaning, one flag is the same as one beam, meaning they have the same duration. The beam simply makes it less visually crowded when reading. The omitting of beams and only using flags *can* mean that the notes are more separated, but this is not neccessarily the case.

2.2.3 Scales

The musical scale is building block for creating not only melodies but also harmony, which we will cover a bit later. Firstly the musical scale is, for the most part, constructed using seven notes. In the western canon this is usually seven notes. You can see the basic c-major scale in Figure 2.2.1, it is comprised of the notes c d e f g a b c. The tones of the scale can be named according to their function within the scale. We often label the notes $\hat{1}$, $\hat{2}$, $\hat{3}$, $\hat{4}$, $\hat{5}$, $\hat{6}$, $\hat{7}$, and then starting back at $\hat{1}$. The use of the carets are called the Nashville Number System, this is used to differentiate the scale degrees from the chord tones which will be

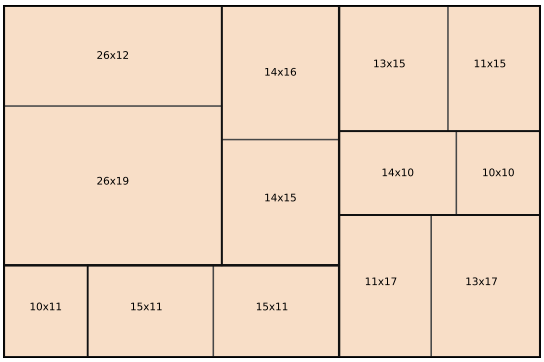


Figure 2.1.10: Initial domain partitioning, made by recursively divinging the domain in two.

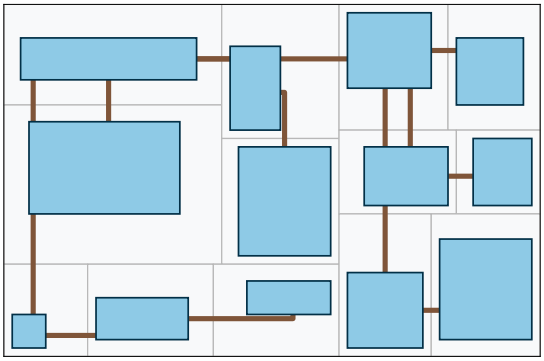


Figure 2.1.11: Generated rooms inside the partitions and connected them with corridors between rooms.

discussed in Section 2.2.7 When we consider chord structures we use the numbers but usually count above from 8 to signify harmonic functions, but when considering only scales and melodies we usually only use 1 through 7

The most important function is that of the scale degree 1, this is the *tonic*, which is the 'home' of the key. This is the most stable function there is, it feels resolved and does not have any tension urging it to move anywhere.

Next up we have the dominant function, there are two notes degrees which fall into this category, that is the 5th and 7th scale degree. These notes have tension. They want to move, especially the 7th scale degree, it has a great pull toward the tonic. You can hear this in the example under Example 2.2.1. The penultimate tone is the 7th scale degree and it leads to the tonic.

To lead us to the tonic functions we have what are called the pre-dominant function. This simply means 'before dominant', and their function is to set up the tension to lead us to the dominant degrees which in turn leads us back to the tonic.

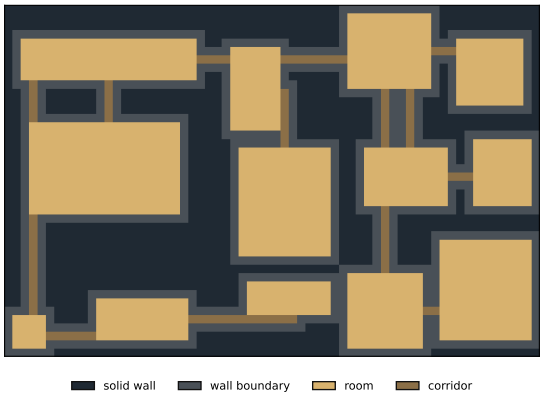


Figure 2.1.12: Rasterization of the generated dungeoun to show "final product".

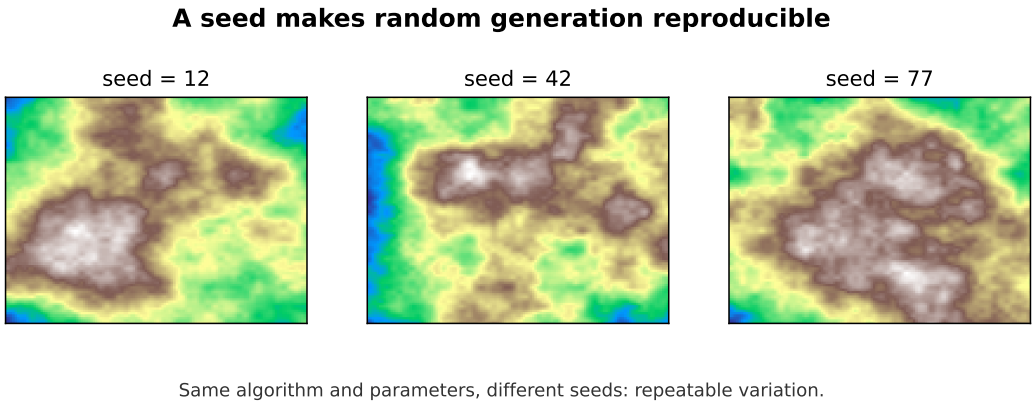
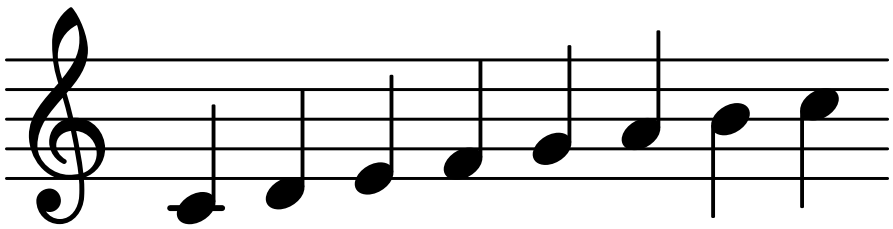
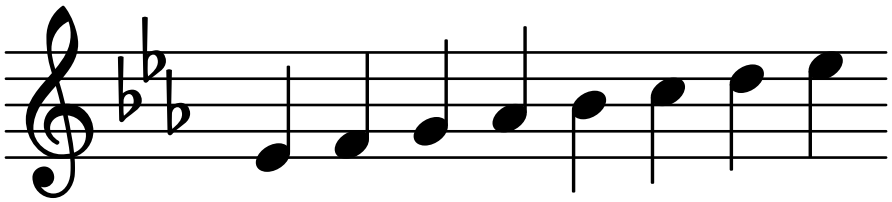


Figure 2.1.13: A seed makes random generation reproducible.



Example 2.2.1: The C-major scale written in the treble clef.



Example 2.2.2: The E♭-major scale written in the treble clef.

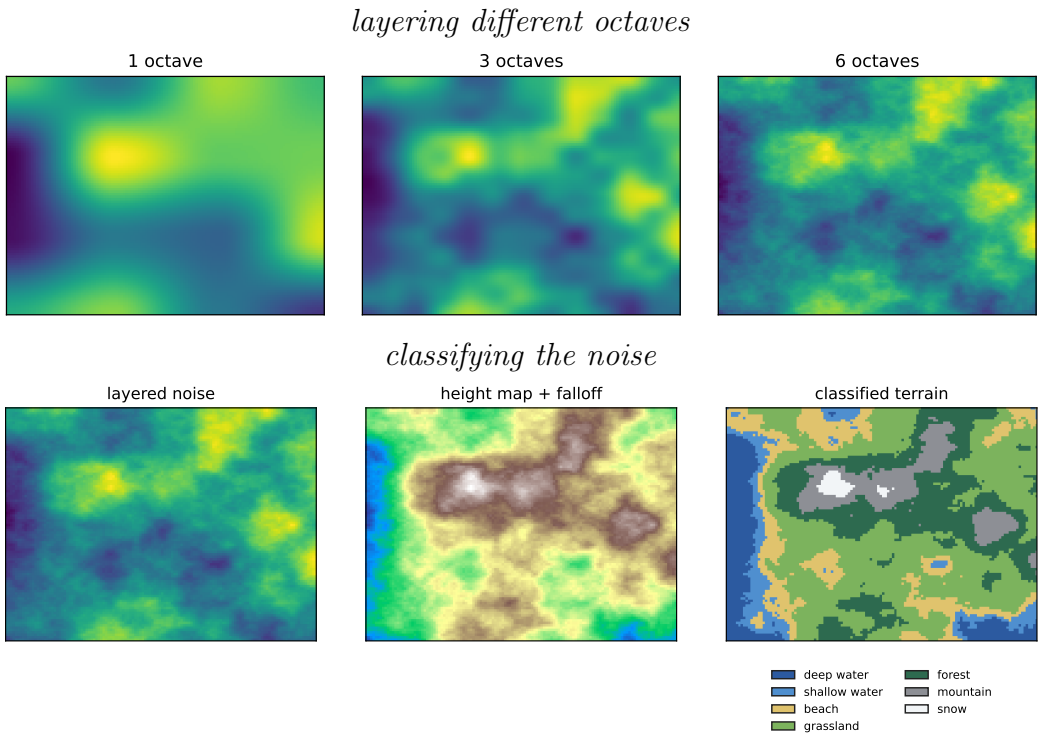


Figure 2.1.14: Using different octaves we can combine lower and higher frequency textures to create something more complex. Using this layered noise we can classify the noise and make it into terrain.

2.2.4 Keys

The concept of a musical key is simply shifting around where the tonic note is. For instance in Example 2.2.1 you can see the major scale in C. And in Example 2.2.2 you can see the major scale in E♭. In todays tuning system there are no differences between the keys, meaning that the sound the same, just higher and lower. This was not the case earlier in history, where the temperament(how the tones were tunes in relation to each other) of the keys differed. Today it just sounds higher or lower depending on the key.

2.2.5 Melodies

Melodies is the horizontal component of music, meaning we look at it as a time-series. We have a given note at a given length followed by another note or rest of a given length etc... This means that you can look at a melody as a series of intervals as well. If we do this on known melodies we will observe that they are mainly composed of steps and skips. This means that the distance between one note and the next is one or two tones. Large leaps are usually leaping to important tones like the tonic or the root of the current chord.

2.2.6 Intervals

Before discussing harmony we need to consider the different types of distances between notes. The musical distance between two notes are called an *interval*.

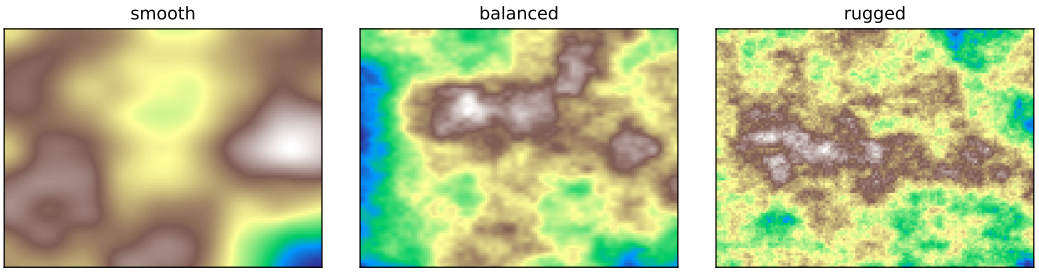


Figure 2.1.15: Using the same seed with different parameters we can control the identity of the output like style and scale.

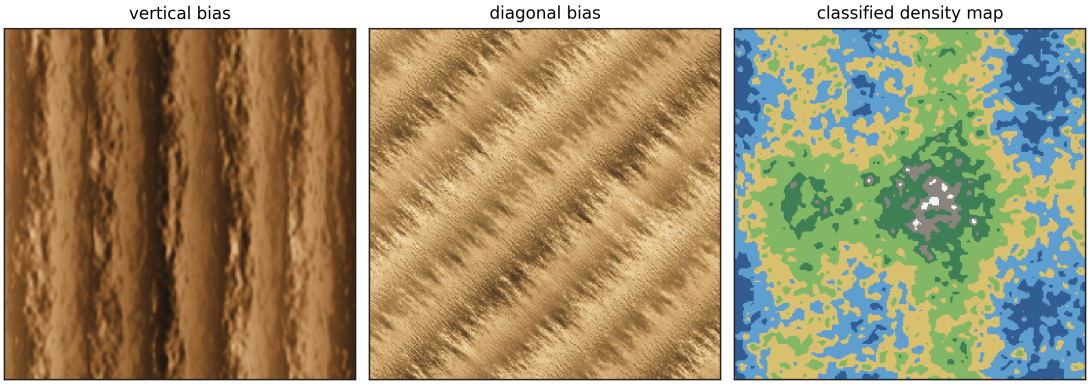


Figure 2.1.16: Three different textures generated using random walks.

Intervals can be identified by the number of semitones between the notes. Additionally we have special names for all the intervals, these names can be seen in Table 2.2.2. The list also provides the number of semitones between the notes.

In Figure 2.2.3 we can see how all the normal intervals look on a stave. Naming the interval on a general basis means looks at the distance, this means that a second is two consecutive notes and a third has "one space" between the notes, this is regardless whether it is minor or major a third looks the same distance wise regardless of quality. You can see this in Example 2.2.3, here we observe that the distance between the notes in a minor third and a major third is the same, the only difference is that the minor third is one semitone smaller, but that is shown by the

Table 2.2.1: Table with common note values, their names and their corresponding duration.

Symbol	Name	Rest symbol	Duration
𝅝	Whole note	𝄻	4 beats
𝅗𝅥	Half note	𝄼	2 beats
♩	Quarter note	𝄽	1 beat
♪	Eigth note	𝄾	1/2 beat
𝅘𝅥𝅯	Sixthteenth note	𝄿	1/4 beat
𝅘𝅥𝅰	Thirtysecond note	𝅀	1/8 beat

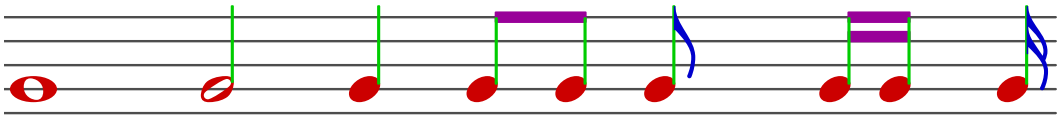


Figure 2.2.1: The figure shows the parts of the musical note. The note-head in red, stem in green, beam in purple and flag in blue.

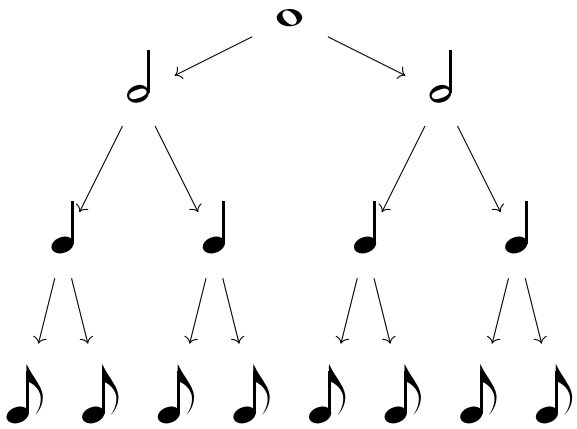


Figure 2.2.2: Tree sctructure showing how you can divide the whole note into the smaller note values. Here each level in the tree takes the same amount of time, meaning the whole note at the top takes as long as the eight eigth-notes at the bottom.

♭-symbol which means the note has been lowered by a semitone. Consequently for the tritone we see there are two ways to spell the same interval, one is a fourth that has been raised and the other is a fifth that has been lowered. These two are *enharmonically equivilant* which just means that they are the same pitches, they are just spelled differently. For the augmented fourth(the first spelling of the tritone) we se the ♯-symbol used, which simply means the note is raised by a semitone.

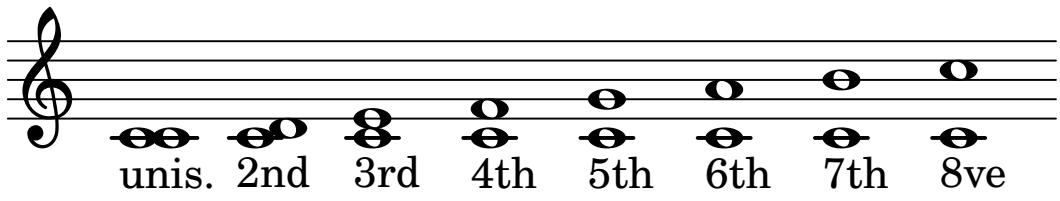
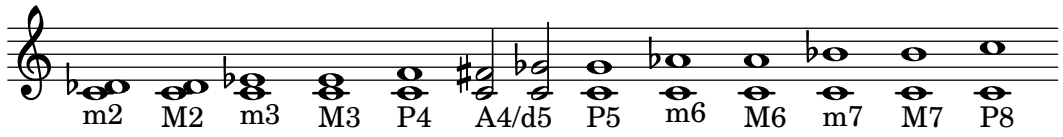


Figure 2.2.3: All normal intervals from the note C_4 devoid their variants in quality.

[3]can also be called diminished fifth or augmented fourth, where diminished simply means lowered by semitone and augmented means raised by one semitone

Table 2.2.2: Table containing all musical intervals up to an octave, the number of semitones and their abbreviated form.

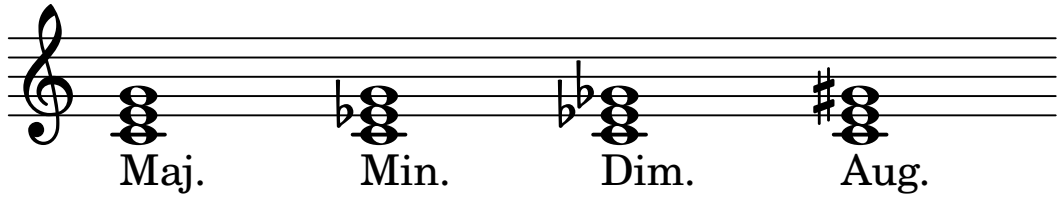
Interval name	Semitones	Abbreviation
Unison	0	P1
Minor second	1	P1
Major second	2	m2
Minor third	3	m3
Major third	4	M3
Perfect fourth	5	P4
Tritone[3]	6	TT or aug4/dim5
Perfect fifth	7	P5
Minor sixth	8	m6
Major sixth	9	M6
Minor seventh	10	m7
Major seventh	11	M7
Octave	12	P8



Example 2.2.3: All the musical intervals from the unison to the octave including their variants.

2.2.7 Harmony

Harmony is the vertical component of the music, harmony is what happens when two or more notes happen at the same time. This is what chords are build around. When considering the harmonic progression in tonal music we can consider all the standard chords in the major key. They are labeled by numbers using the roman numerals. Upper case means the chord in major and lower case means minor. You can see the difference between major and minor chords in figure [make a figure for this], and listen to the difference at the link provided in the caption.



Example 2.2.4: The four main triads(chords with three notes), major, minor, diminished and augmented.

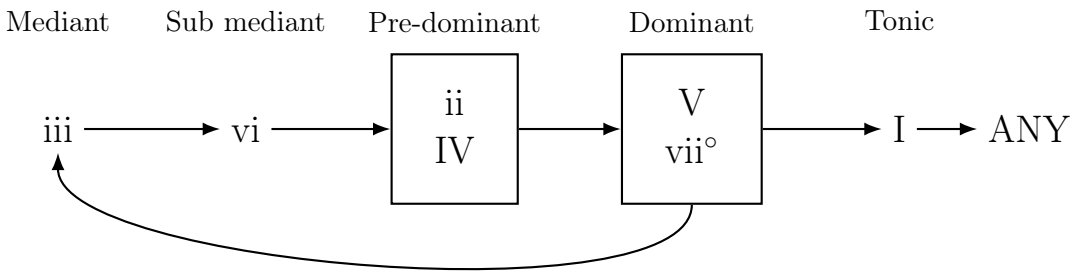


Figure 2.2.4: Harmonic progression in tonal music. The arrows indicate the typical flow of harmonic functions.

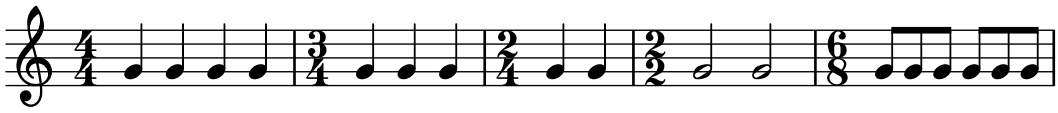


Figure 2.2.5: Demonstration of a few common time signatures.

2.2.8 Time signatures

The time signature explains what is in a bar of music. A bar is the space between the vertical lines on the staff. A 4/4, sometimes written as **C**, time means that there are four quarter notes per bar and consequently 3/4 time means that there are three quarter notes per bar.

2.2.9 Musical form

Now that we have an idea of the musical structures on the melodic and harmonic layer we will take a look at musical forms. This is a very important idea when it comes to constructing melodies. A melody is not an infinitely wandering idea of music. A good melody follows a form. Often this can be viewed as repetitions and variations of a given musical idea. Two very common forms of musical form are the *sentence form* and the *period form*.

In Figure 2.2.6 we can see an example of the musical sentence. It starts with a 2-bar opnening phrase, labeled as the *basic idea*, then we have a repetition of the same idea only adapted for a new chord. Then the following 2 bars are a fragmentation of the idea. Then for the last two bars we have a conclusion, this ties the idea together. The cadence at the end can be considered the end of a sentence. It concludes the idea before it moves on to something else.

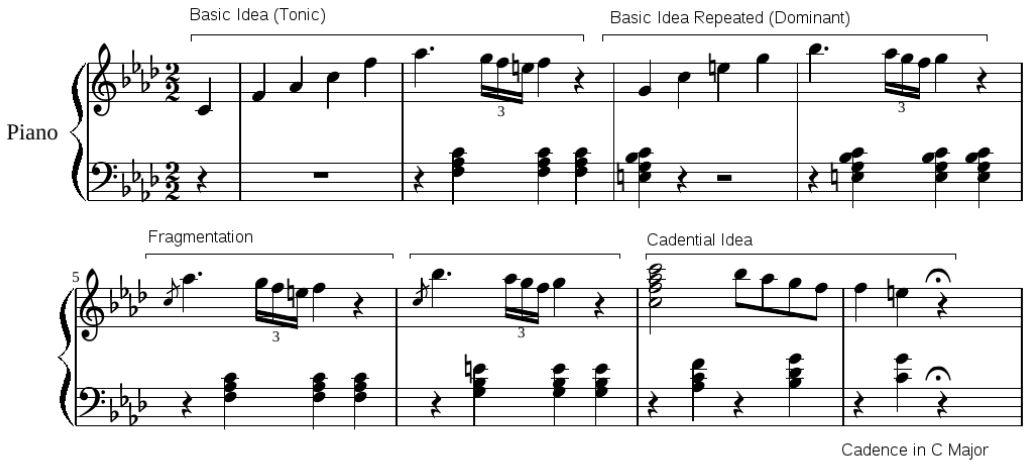


Figure 2.2.6: A classical example of the musical sentence form. These are the opening bars of Bethovens piano sonata in F-minor. [3]

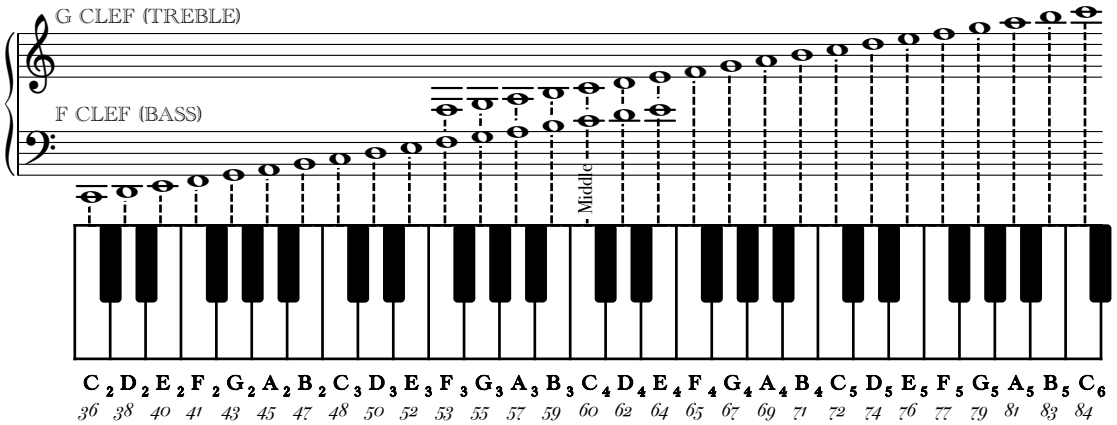


Figure 2.2.7: Overview of note placement on the staff and piano, additionally the corresponding midi numbers are written below the note names. Notice how the number increases by two when there is a black key between consecutive notes.

2.3 Object oriented programming

OOP had its inception in the sixties with the *aritficial intelligence* group at Massachusetts Institute of Technology (MIT). Here the term "object" was used in the language family List Processing (LISP) to refer to LISP atoms[4] with with certain attributes or properties. [4]

2.3.1 Features of OOP

Some of the useful features of OOP are that we can achieve using OOP is ...

[4]a LISP atom is an indivisible entity in the language. LISP consist of two data types, atoms and lists. And a list can only contain atoms or other lists

2.4 Iterative development

> A major problem in the design of a computer system is that many aspects of system behaviour and performance are not discovered until the system has been built...[5]
>
> *F.W. Zurcher*
> *B. Randell*

This project utilizes a from of development called iterative design. This means that I do the project by creating a design, taking notes on what needs to be improved, then starting again with the new knowledge gained in the previous iteration.

This sounds like more work, but in reality it leads to an effective workflow. It allows for fast prototyping since we can start programming something straight away, then when the project halts due to the system that has been created is incompatible with the ideas for improving it we take notes on what worked, what is missing, and use this to create a specification for the next iteration.

By doing this we maintain active development which doesn't stagnate and we can always create a new iteration when development slows down. After 3-4 iterations we should be able to land on a system that works in a way that we are happy with.

2.5 AI in software development

In addition to doing this iterative development we will attempt to develop using artificial intelligence as a tool for assisting the process. However the first two iterations will be without any AI. This is to make sure I understand the domain and make my own decisions of how I want the program to work. By doing this work to begin with I work up knowledge in the field and will be better equipped to understand what the AI procudes and make sure that it does what I want it to. Without this knowledge I will not be able to efficienlty grasp the AI output and will spend a significant time learning what the AI has done.

I will discuss the use of AI and whether it is a tool worth using or if it simply ends up creating more work since we need to manually fix bugs or do prompt engineering

CHAPTER

THREE

SOFTWARE DESIGN

Include the complete description of the methods used in your research here.

hva skal med i denne seksjonen

3.1 Object oriented programming

This project will use the widely known process of object oriented programming. In simple terms this simply means that all the parts of the code are structured as objects, meaning everything is represented as objects with given properties. This means that for instance a 'melody object' could consist of all the notes and durations it consists of, the key it is written in and any metadata explaining what the melody represents. Meaning if it is a verse or chorus etc...

In order to design an object oriented program the design process is very important. Creating good models for the objects are imperative to make it work. If the level of abstraction is too high, the sematic gap between the real-world concepts they represent and the acutal implementation in the code will be too large and the model will be un-intuitive and hard to both use and also understand.

3.2 Final iteration design?

CHAPTER

FOUR

THE FIRST ITERATION

The implementation of this program started by just writing something that generated something. This was done to not get stuck in the planning phase. You could consider this a contradiction to the part in software design which states that good planning is imperative. However, trying out some code and figuring out what works, what doesn't and what needs to be implemented in the models is very helpful. Therefore the first iteration was a test to get some systems working and finding out what it neccesary to make it work and what is not imporatant.

4.1 The iteration process

After the first iteration lots of issues were observed and were put into the spec for the second iteration which improved upon the model in various ways. Some imporatnt features were restricting the range so that an acutal instrument can play it, be that an instrument or the human voice.

The goal of the first iteration was to get a "proof of concept" working. I wanted to test out how I could generate notes and visualize them as well as listen to audio of the notes generated. For this I used *python* to generate *liypond* files. From these files I was able to generate PDFs of sheet music as well as *wave-files* to listen to how it sounds without having to play/sing it myself.

4.2 Software architecture

Figure 4.2.1 shows the structure of the first iteration. The pipeline is small and direct: the configuration is hard-coded in the main script, rhythm and pitch are generated independently for each voice, and the result is exported directly to LilyPond without a richer planning or constraint layer.

4.3 Implementation

This first iteration was a very primitive handling of generating music. It had very few constraints on the melody and contained little to no metadata on the music which made it unsuitable for actual music.

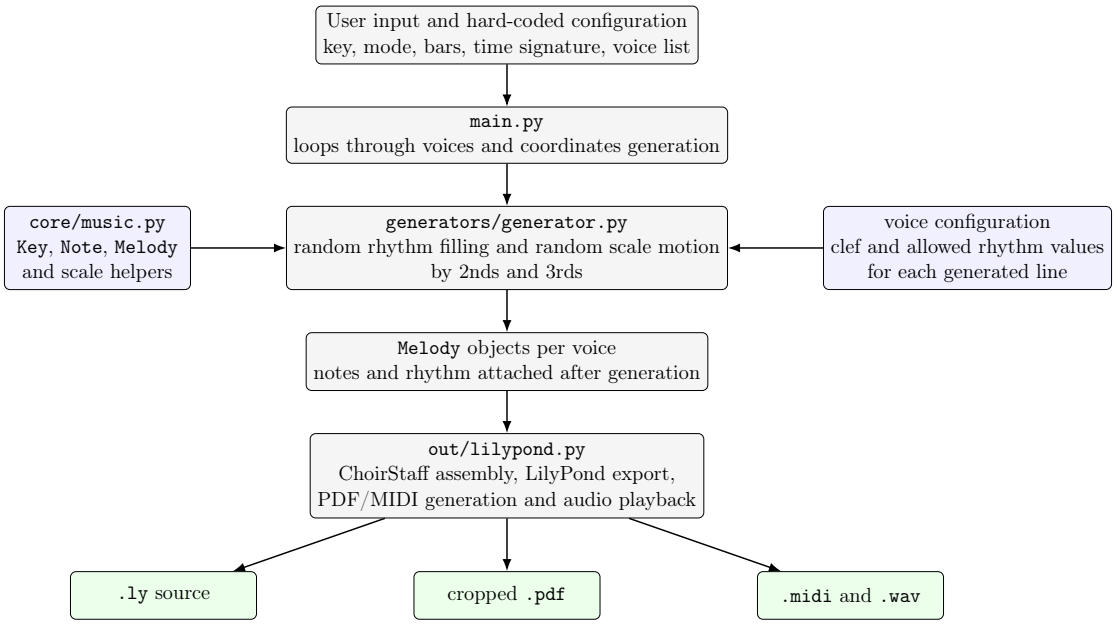


Figure 4.2.1: Architecture of the first iteration melody generator. The first proof of concept uses a small script-driven pipeline where rhythm and pitch are generated per voice and then exported directly through LilyPond.

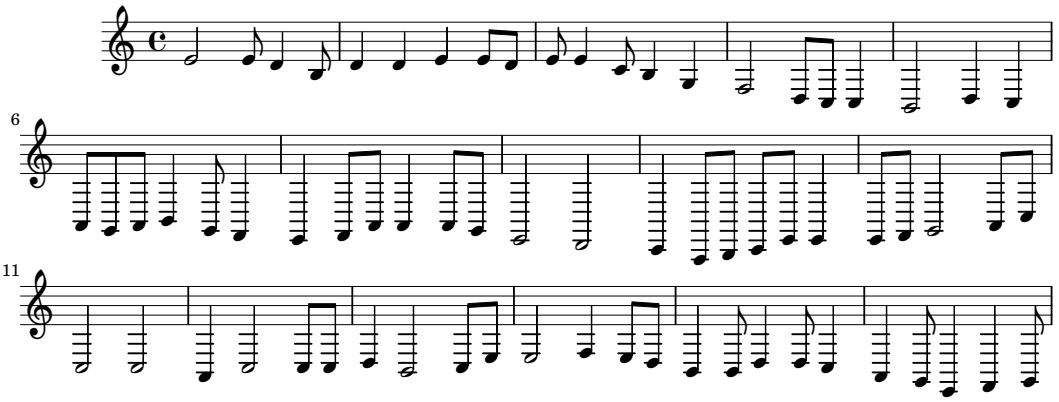
The code generated the notes within a given scale and restricted the distance (called interval in music theory) to only 2nds and 3rds. However this meant that the melody meandered without any other restrictions. If the generated music was long it would have a tendency to go beyond the range of an instrument or the human voice.

4.4 Results

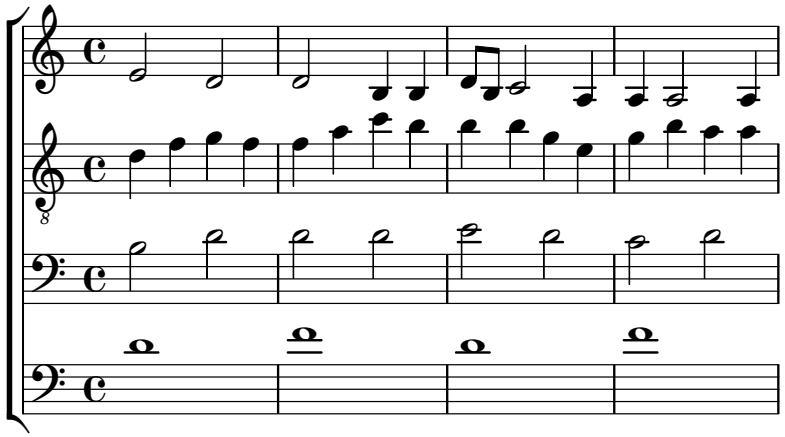
The melody generation works fine for smaller melodies, however there is no self-similarity and no rests. This means that the melodies we generate are not particularly musical or interesting, but it is a great starting point to getting to a fully fledged melody generator. In Example 4.4.3 we can see how a short melody using the first iteration looks.

In Example 4.4.1 you can see what happens when the generated melody is longer, it will tend to go beyond what an instrument will be able to play. You can see this clearly in *bar 6 through 16* (the entire 2nd and 3rd system), where there are very long ledger lines. Some of these notes are not even available on the grand piano.

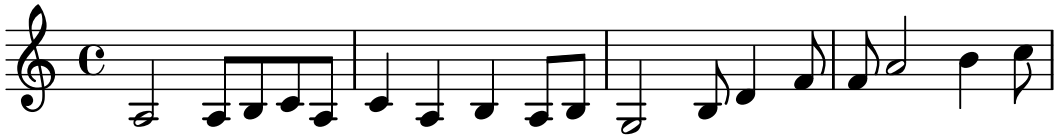
However a good feature that was included was the ability to generate more voices at the same time. This feature was quite primitive since the voices were generated independently and did not adhere to any harmony, however it can be a good skeleton to use later if we get to the point that we want to actually harmonize with the melody we generate. In Example 4.4.2 you can see how we generated four voices all with their own rhythmic restriction. The bottom voice only has whole notes, the second lowest one has only half-notes, the second highest has only quarter and the top voice has a combination of several types.



Example 4.4.1: Melody showing how the octave is not restricted and ends up in unplayable range.



Example 4.4.2: Melody showing how we can generate multiple voices with different rhythmic restrictions.



Example 4.4.3: Example melody of the first iteration melody generator.

4.5 Discussion

As we can see the first iteration shows a promising proof of concept, however it is quite limited and needs lots of improvement before we have a good system for melody generation.

For the second iteration we need to be able to enforce more restictions on the melody so that it follows more convetions of good melody writing. Additionally we need to have some self-similarity so that the music follows some kind of musical form in order for it to feel coherent and part of the same idea.
The second iteration will focus on creating a generation algorithm that tat is able to generate a melody with a structure. Meaning when generating a melody we will generate a phrase, that phrase should contain an idea, a repetition, and the end of the melody should end resolved with a long note on the tonic of the key. Additionally we need to restirct the range of the melody so that it will be playable/singable.

CHAPTER

FIVE

THE SECOND ITERATION

As discussed in the previous section there are several issues with the first iteration the second iteration aims to solve. Most of the improvements over the first iteration are not seen by the end-user as the bulk of the improvement is the code handling the musical objects.

5.0.1 Software architecture

Figure 5.0.1 shows the second iteration in the same architectural style as the other iterations. Compared with the first iteration, the generation now passes through explicit musical objects such as `Motif`, `Bar`, and `Phrase`, and the generator reuses motivic material instead of producing every event independently.

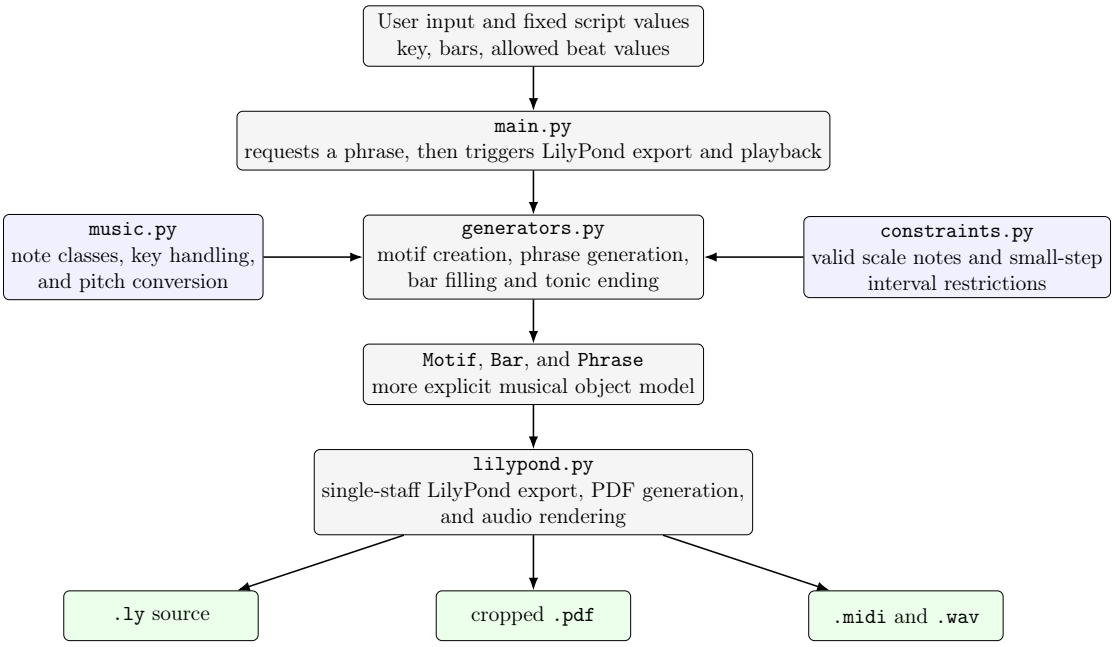


Figure 5.0.1: Architecture of the second iteration melody generator. Motivic material and phrase-level objects make the generator more structured than the first iteration, even though the constraint handling is still lightweight and local.

5.1 Implementation

As mentioned the code structure has been improved in this iteration, below you can see an excerpt form the code in the *music.py* file.

```
class Note:
    def __init__(self, degree, key, midiNum, duration):
        self.degree = degree
        self.key = key
        self.midiNum = midiNum
        self.duration = duration

class Motif:
    def __init__(self, key, beats=[], notes=[]):
        self.key = key
        self.beats = beats
        self.notes = notes

    def getLength(self):
        length = 0
        for beat in self.beats:
            length += beat
        return length

class Bar:
    def __init__(self, key, beats, notes=[]):
        self.beats = beats
        self.notes = notes

class Phrase:
    def __init__(self, key, bars):
        self.bars = bars
        self.key = key
```

Comparing to the first iteration this has a more defined structure and easier to read, however it needs improvement.

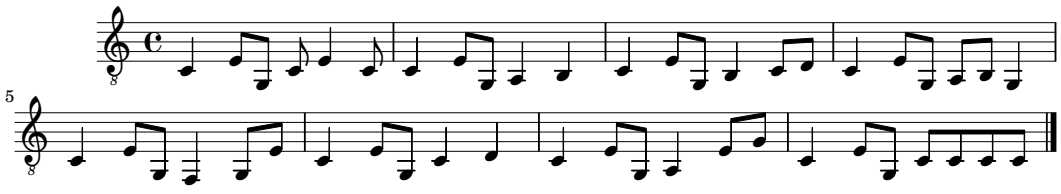
The second iteration improved upon the structure of the melody. By using a motif we forced some self-similarity in the generated melody which helped generating something more musical, however this generation is very dependent on the generated motif being musical.

Structurally the second iteration is quite similar to the first iteration however, the objects used for handling the melodies and such are more robust and easier to understand. However they are not in a final state where we will re-use it in its entirety for the next iteration.

5.2 Results

We can see in Example 5.2.1 that we have a more defined structure. The generator works by generating a motif which is then re-used in every measure in the piece.

The rest of the measures are filled with random notes moving by 2nds or 3rds to make a somewhat coherent and singable melody.



Example 5.2.1: Demonstration of a melody generated using the second iteration.

The generated melodies are still quite random and does not follow any musical form or adhere to melodic restrictions other than that the melody has smaller steps and not big leaps.

5.3 Discussion

The most glaring issue with this iteration is that the melodic rules have not been formalized and implemented as code. Generating a melody using this restricted form of generation works well when the generated motif is good, and with the current implementation this only happens randomly. By implementing melodic rules and restrictions this generator will greatly improve the melody generation.

In the first iteration also had the ability to generate several voices at the same time, however, this will not be re-implemented until the melodic generation has been improved and we can enforce harmonic restrictions on one voice. If this is achieved then multi-voice generation will become much more interesting.

Additionally the second iteration does not properly spell all notes in the different keys, this does not affect the generation in any way, though it is a trivial fix it was not implemented since the structure of the second iteration was incompatible with it. Therefore it will only be implemented in the final iteration.

We would also like to implement augmentation of a given melody or phrase, meaning the ability to transpose it up or down or use the same rhythmic outline and moving the notes around to follow a new chord. This will help develop the generated motif and melody to follow a more structured outline of a well composed melody. A key to a good melody is self-similarity, the second iteration just has repetition of a motif, not variations of it.

The main goals of the third iteration is to be able to augment already generated melodies and be able to enforce constraints on the generation. To ble able to easily augment the melodies and phrases we need a robust model for the melodies made of good objects. Well designed objects is also key to ble able to enforce constraints.

CHAPTER

SIX

THE THIRD ITERATION

For the third iteration the use of AI was implemented.

6.1 Software architecture

In Figure 6.1.1 we can see how the system is built and how the different modules interact. As we can see this is a much bigger and complicated system than the first two iterations already.

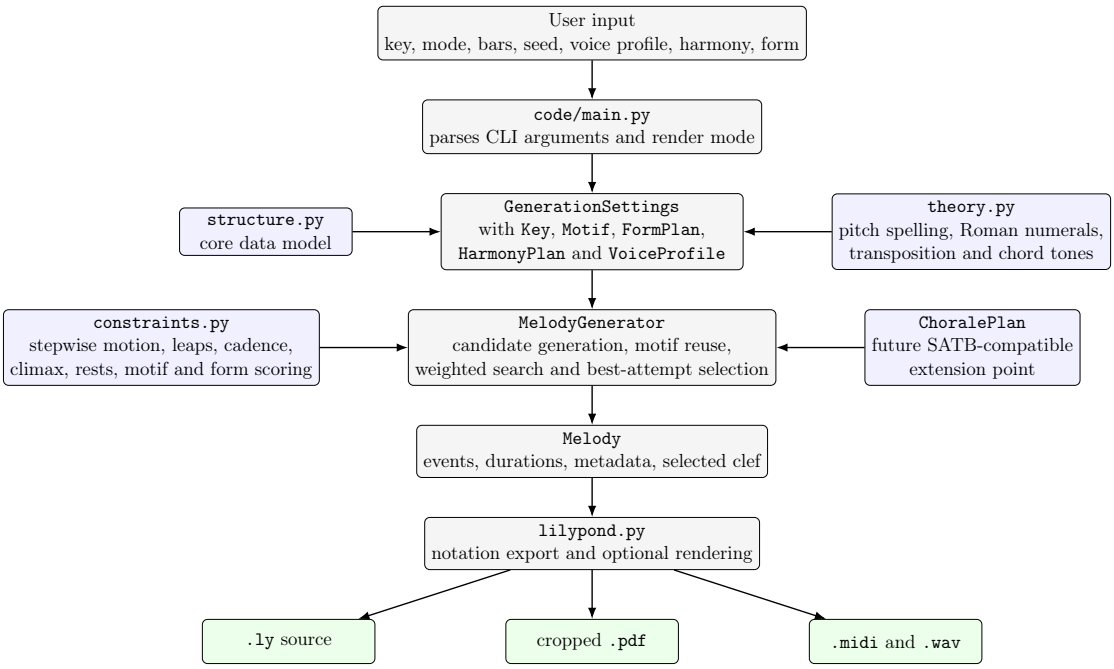


Figure 6.1.1: Architecture of the third iteration melody generator. The command-line layer builds a structured settings object, the generator combines theory-aware candidate generation with a soft-constraint stack, and the resulting melody is exported through LilyPond into notation and audio assets.

CHAPTER

SEVEN

DISCUSSION

7.1 Iterative development

———————— re-read all of this ———————— The iterative development process has been very useful. It has enabled me to use a few iterations to get to know the problem before trying to create the final solution. If I had started trying to solve the problem completely straight away I would have ended up with a very hap hazard solution.

Figure 7.1.1 shows how the three iterations grew over time. The first iteration reached a moderate code size quickly because it solved a small but concrete proof-of-concept problem: generate notes, render them in LilyPond, and optionally generate several independent voices. The second iteration started from a much smaller base and stayed smaller overall because it focused on improving the internal representation and introducing motif-based generation rather than adding many new subsystems. The third iteration grows much more rapidly because it combines a richer data model, explicit constraint handling, harmony support, form planning, rendering workflows, and later SATB-compatible extensions.

This development pattern is important because it shows that the final system did not appear fully formed. Instead, each iteration solved a narrower version of the problem and exposed the next set of weaknesses. The first iteration showed that note generation and rendering were feasible, the second showed that musical reuse required better abstractions, and the third was where the musical rules and architectural structure were finally formalized in a way that could support a larger system.

Table 7.1.1 summarizes the practical differences between the iterations. The main observation is that the first two iterations each solved only part of the problem. Iteration one had rendering and even rudimentary multi-voice support, but the generated melodies were musically weak. Iteration two improved motivic handling and made the code structure easier to reason about, but it still lacked explicit theory-aware constraints. Only the third iteration combines musical structure, reusable abstractions, harmony handling, and a workflow that is robust enough to support the thesis examples.

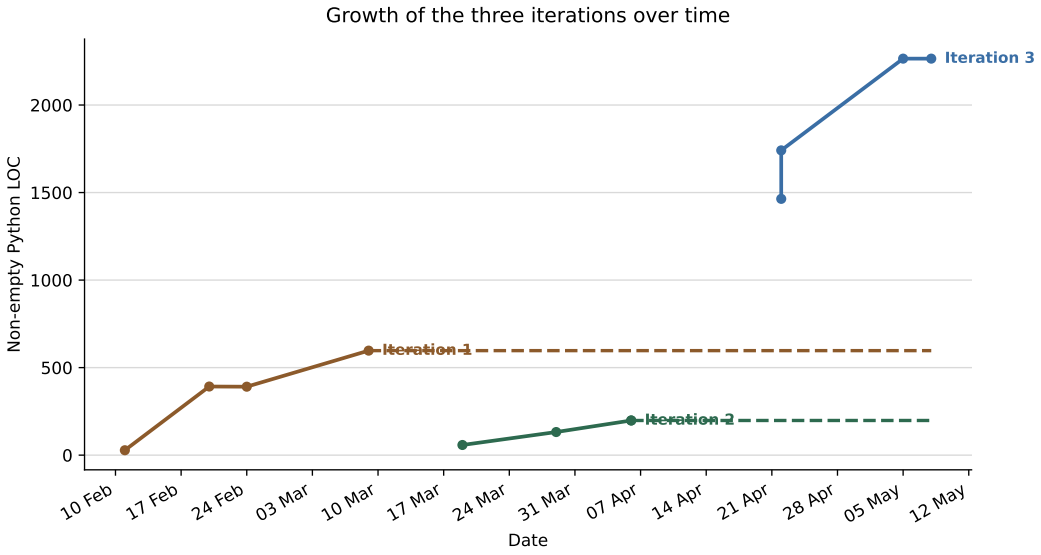


Figure 7.1.1: Growth of the three iterations over time measured as non-empty Python source lines in the iteration-specific code. Dashed segments indicate that an iteration had been frozen while the thesis work continued.

7.2 AI development

After the first few iterations I had build up a better understanding of not only the problem, but also how I should go about solving it. Having this in the back of my mind when prompt engineering for the AI to help me solve it make the end result a lot better than if I tried to solve it using AI from the very beginning.

7.3 The system structure

The architectural progression is also visible when comparing the system diagrams from the three iteration chapters. Figures 4.2.1, 5.0.1, and 6.1.1 show a clear movement from script-driven generation toward a more modular and theory-aware design.

Table 7.3.1 complements the architecture figures by showing what changed conceptually between the iterations. The most important difference is not just that the third iteration has more modules, but that it has more explicit musical objects and clearer boundaries between responsibilities. This makes it possible to reason about melodic constraints, harmony, rendering, and future extensions separately rather than treating the generator as one monolithic script.

7.4 Modules? Different module discussions.

7.5 The generated music

7.6 Future work

Feature	Iteration 1	Iteration 2	Iteration 3
LilyPond export and audio rendering	yes	yes	yes
Independent multi-voice output	yes	no	yes
Explicit motif reuse	no	yes	yes
Phrase or form planning	no	limited	yes
Automatic tonal harmony plan	no	no	yes
Roman-numeral harmony input	no	no	yes
Melodic soft constraints	no	limited	yes
Rest handling in melody generation	no	no	yes
Register-aware voice profiles and clefs	no	no	yes
SATB-compatible harmonization path	no	no	yes
Descriptive CLI and reproducible seed workflow	limited	limited	yes

Table 7.1.1: Comparison of the main musical and technical capabilities across the three iterations.

Aspect	Iteration 1	Iteration 2	Iteration 3
Main representation	note and melody objects with limited metadata	motif, bar, and phrase objects	settings, motif, form, harmony, melody, and chorale-related objects
Generation strategy	random local note motion with separate rhythm generation	motif-first phrase generation with small-step filling	weighted search with explicit candidate scoring and harmonic context
Harmony handling	none	none	manual Roman numerals or automatically generated tonal plan
Module structure	small script plus helper modules	clearer separation, but still tightly coupled	dedicated theory, structure, constraints, generator, rendering, and CLI layers
Extensibility	difficult to extend without rewriting	better than iteration 1, but still narrow	designed to support variants, rendering modes, and future multi-voice work

Table 7.3.1: Architectural comparison of the three iterations.

CHAPTER

EIGHT

CONCLUSION

This project has shown the development of a procedural music generator. The process utilized iterative development as well as AI assitance with developing the system. This resulted in a modular system which encapsultes musical ideas in an effictive manner which is easy to understand and modify.

The use of AI proved to be a very useful tool to speed up development both in terms of troubleshooting or debugging as well as helping find solutions to the problems you are working on. The issue with AI is that is removes the creative element of programming and simply regurgitates from the ocean of information it has access to. This means that it solves the problems in similar ways they have been done before, this hinders true innovation and creating something completely new.

On the other end we have the problem that AI is very good at generating content, meaning by prompting AI do solve a problem for us it generates a lot of code, this means that in addition to formulating the problem in such a way that AI can solve it we also need to go over the code and check that it actually does what we inteded it to as well as it being maintanable and easy to read. In order to to this part effectively we need to understand now only the language we are developing in, but we also need knowledge of the existing code base. For this problem it means that we need to know not only how the language works, but also the language of music theory. If we do not understand this then we would not be able to make any meaningful changes to the code by ourselves. Let alone if we try to prompt engineer without knowing anything about music.

There seems to be a sort of silver lining in the use of AI when developing software. Using it to develop entire systems will lead to the developer not understanding how the code works or having to spend a substantial amount of time reading through the AI generated code to figure out how it works. Using AI in smaller perspectives can yield better results. It is very good at saving you time by completing trivial tasks that just takes time or debugging small funcitons etc. that you could have spend minutes or hours tinkering with to solve yourself. This keeps the code your own, but it makes you faster by helping you with the small tasks.

Overall the system generates passable music, there is no defined way to actually grade music objectively, but at least we know that the output follows a few

principles associated with good melodic and harmonic structure.

All references to check formatting... [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [1] [4] [20] [21] [22] [3] [23] [24] [25] [26] [5]

REFERENCES

[1] *Musikalisches Würfelspiel*. In: *Wikipedia*. Sept. 27, 2025. URL: https://en.wikipedia.org/w/index.php?title=Musikalisches_W%C3%BCrfelspiel&oldid=1313693497 (visited on 03/24/2026).

[2] *Arc-Consistency Algorithm - an Overview | ScienceDirect Topics*. URL: https://www.sciencedirect.com/topics/computer-science/arc-consistency-algorithm (visited on 05/08/2026).

[3] *Sentence (Music)*. In: *Wikipedia*. June 21, 2023. URL: https://en.wikipedia.org/w/index.php?title=Sentence_(music)&oldid=1161220566 (visited on 03/26/2026).

[4] *Object-Oriented Programming*. In: *Wikipedia*. Mar. 23, 2026. URL: https://en.wikipedia.org/w/index.php?title=Object-oriented_programming&oldid=1344935717 (visited on 03/27/2026).

[5] F. Zurcher and Brian Randell. "Iterative Multi-Level Modeling - A Methodology for Computer System Design". In: Jan. 1, 1968, pp. 867–871.

[6] *[1809.04281] Music Transformer*. URL: https://arxiv.org/abs/1809.04281 (visited on 02/26/2026).

[7] Gabriel Aguilera et al. "Automated Generation of Contrapuntal Musical Compositions Using Probabilistic Logic in Derive". In: *Mathematics and Computers in Simulation*. Fifth IMACS Seminar on Monte Carlo Methods 80.6 (Feb. 1, 2010), pp. 1200–1211. ISSN: 0378-4754. DOI: 10.1016/j.matcom.2009.04.012. URL: https://www.sciencedirect.com/science/article/pii/S0378475409001165 (visited on 02/26/2026).

[8] Torsten Anders and Eduardo R. Miranda. "Constraint Programming Systems for Modeling Music Theories and Composition". In: *ACM Comput. Surv.* 43.4 (Oct. 18, 2011), 30:1–30:38. ISSN: 0360-0300. DOI: 10.1145/1978802.1978809. URL: https://dl.acm.org/doi/10.1145/1978802.1978809 (visited on 02/26/2026).

[9] Ye Bai et al. *Seed-Music: A Unified Framework for High Quality and Controlled Music Generation*. Sept. 19, 2024. DOI: 10.48550/arXiv.2409.09214. arXiv: 2409.09214 [cs]. URL: http://arxiv.org/abs/2409.09214 (visited on 03/24/2026). Pre-published.

[10] Prafulla Dhariwal et al. *Jukebox: A Generative Model for Music*. Apr. 30, 2020. DOI: 10.48550/arXiv.2005.00341. arXiv: 2005.00341 [eess]. URL: http://arxiv.org/abs/2005.00341 (visited on 02/26/2026). Pre-published.

[11] Lucas N. Ferreira and Jim Whitehead. *Learning to Generate Music With Sentiment*. Mar. 9, 2021. DOI: 10.48550/arXiv.2103.06125. arXiv: 2103.06125 [cs]. URL: http://arxiv.org/abs/2103.06125 (visited on 03/24/2026). Pre-published.

[12] E. Galin et al. "Procedural Generation of Roads". In: *Computer Graphics Forum* 29.2 (2010), pp. 429–438. ISSN: 1467-8659. DOI: 10.1111/j.1467-8659.2009.01612.x. URL: https://onlinelibrary.wiley.com/doi/abs/10.1111/j.1467-8659.2009.01612.x (visited on 02/26/2026).

[13] Andres Garay Acevedo. "Fugue Composition with Counterpoint Melody Generation Using Genetic Algorithms". In: *Computer Music Modeling and Retrieval*. Ed. by Uffe Kock Wiil. Berlin, Heidelberg: Springer, 2005, pp. 96–106. ISBN: 978-3-540-31807-1. DOI: 10.1007/978-3-540-31807-1_7.

[14] Maxim Gumin. *Wave Function Collapse Algorithm*. Version 1.0. Sept. 2016. URL: https://github.com/mxgmn/WaveFunctionCollapse (visited on 03/24/2026).

[15] Curtis Hawthorne et al. *Enabling Factorized Piano Music Modeling and Generation with the MAESTRO Dataset*. Jan. 17, 2019. DOI: 10.48550/arXiv.1810.12247. arXiv: 1810.12247 [cs]. URL: http://arxiv.org/abs/1810.12247 (visited on 02/26/2026). Pre-published.

[16] Johan Gangsås Hole. "Automatic Species Counterpoint". In: *Master Thesis at NTNU* (2026).

[17] Hao Hua. "A Bi-Directional Procedural Model for Architectural Design". In: *Computer Graphics Forum* 36 (Dec. 1, 2016). DOI: 10.1111/cgf.13074.

[18] Ivar Jacobson et al. *Object-Oriented Software Engineering*. Addison-Wesley, 1992. ISBN: 0-201-54435-0.

[19] John McDonough and Andrzej Herczyński. "Fractal Patterns in Music". In: *Chaos, Solitons & Fractals* 170 (May 1, 2023), p. 113315. ISSN: 0960-0779. DOI: 10.1016/j.chaos.2023.113315. URL: https://www.sciencedirect.com/science/article/pii/S0960077923002163 (visited on 04/07/2026).

[20] Sageev Oore et al. "This Time with Feeling: Learning Expressive Musical Performance". In: *Neural Computing and Applications* 32.4 (Feb. 1, 2020), pp. 955–967. ISSN: 1433-3058. DOI: 10.1007/s00521-018-3758-9. URL: https://doi.org/10.1007/s00521-018-3758-9 (visited on 02/26/2026).

[21] *Period (Music)*. In: *Wikipedia*. Jan. 29, 2025. URL: https://en.wikipedia.org/w/index.php?title=Period_(music)&oldid=1272601348 (visited on 03/26/2026).

[22] *Random Walk*. In: *Wikipedia*. Apr. 6, 2026. URL: https://en.wikipedia.org/w/index.php?title=Random_walk&oldid=1347336606#Applications (visited on 04/09/2026).

[23] Arnold Schoenberg. *Fudamentals of Musical Composition*. 1st ed. faber and faber, 1967. 224 pp. ISBN: 0-571-19658-6.

[24] Adam Summerville et al. *Procedural Content Generation via Machine Learning (PCGML)*. May 7, 2018. DOI: 10.48550/arXiv.1702.00539. arXiv: 1702.00539 [cs]. URL: http://arxiv.org/abs/1702.00539 (visited on 03/24/2026). Pre-published.

[25] Roland van der Linden, Ricardo Lopes, and Rafael Bidarra. "Procedural Generation of Dungeons". In: *IEEE Transactions on Computational Intelligence and AI in Games* 6.1 (Mar. 2014), pp. 78–89. ISSN: 1943-0698. DOI: 10.1109/TCIAIG.2013.2290371. URL: https://ieeexplore.ieee.org/abstract/document/6661386 (visited on 02/26/2026).

[26] Pal Varga and Rafael Bidarra. "Procedural Mixed-Initiative Music Composition with Hierarchical Wave Function Collapse". In: Mar. 26, 2023.

APPENDICES

A - GITHUB REPOSITORY

All code and latex-files used in this document are included in the Github repository linked below. Further explanations are given in the readme-file.

Github repository link

- https://github.com/lima98/TTK4900

B - AUDIO EXAMPLES

Audio-files for the examples in the thesis and code documentation can be found at my github.io page.

Audio example archive

- https://lima98.github.io/TTK4900